Phân tích ngữ nghĩa
Semantics parsing

Định nghĩa

- PTNN là quá trình ánh xạ từ câu NNTN sang dạng biểu diễn logic của nó.
- Mức nông: gán nhãn vai trò ngữ nghĩa (case-role) như agent, patient
- Mức sâu hơn: biểu diễn dạng logic vị từ hoặc hình thức khác cho phép suy diễn tự động.

Tại sao cần phân tích ngữ nghĩa

- Trả lời câu hỏi (bài toán hỏi đáp)
- Hội thoại (chatbot)
- Điều khiển hoạt động của robot
- Dịch máy
- Tóm lược văn bản
- ...

Thế nào được coi là hiểu?

- … nếu có thể phản ứng phù hợp

  VD: “cho tất cả đồ chơi vào giỏ”

- … nếu có thể xác định 1 phát biểu là đúng hay sai

  hiểu NP nghĩa là xác định được NP đó đề cập đến cái gì

- … nếu có thể sử dụng nội dung đó để trả lời câu hỏi

  Dễ: Mai ăn kẹo. => Mai ăn gì?

  Khó: Nước đi đầu tiên của quân trắng là P-Q4. => Quân đen có thể chiếu tướng không?

Thế nào được coi là hiểu?

- … nếu có thể dịch: phụ thuộc vào ngôn ngữ đích
  - Anh – Anh?
  - Anh – Pháp? có thể được
  - Anh – logic? cần hiểu sâu
    - tất cả loài cá đều biết bơi
    - = $\forall x$ [fish(x) $\Rightarrow$can_swim(x)]

Một số khái niệm cơ bản về logic

- 3 loại đối tượng cơ bản:
  - **Giá trị - Booleans**
    - là giá trị ngữ nghĩa của câu
  - **Thực thể - Entities**
    - Giá trị của NP, vd., các đối tượng như bàn, ghế, thời gian
  - **Hàm**
    - Hàm trả về giá trị nhị phân gọi là vị từ (predicate). Vd., frog(x), green(x)
    - Hàm có thể trả về 1 hàm khác
    - Hàm có thể nhận hàm khác như tham số

Logic: thuật ngữ Lambda

- λ:
  - là cách để viết 1 hàm "**bất kỳ**"
    - Không có tên hàm
    - Được dùng để định nghĩa tính chất cơ bản của hàm
  - Cho square = λp p*p
  - Tương đương với int square(p) { return p*p; }
  - Định dạng: λ <tên biến> <biểu thức>

Logic: thuật ngữ Lambda

- λ:
  - Cho square = λp p*p
  - Khi đó square(3) = (λp p*p)(3) = 3*3
  - Định dạng: λ <tên biến> <biểu thức>
  - Chú ý: square(x) không phải là hàm, chỉ là giá trị của x*x.
  - Hàm: **λx** square(x) = λx x*x = λp p*p = square
  - Cho even = λp (p mod 2 == 0) vị từ trả về giá trị True/False
  - even(x) = true nếu x chẵn
  - Còn even(square(x))?
  - λx even(square(x)) = true với các số x có square(x) chẵn
    - λx (even(x*x)) = λx (x*x mod 2 == 0)

Logic: Một số vị từ

- most – 1 vị từ trên 2 vị từ khác
  - most(pig, big) = “most pigs are big”
    - tương đương với, most(λx pig(x), λx big(x))
  - trả về true nếu đa số giá trị thỏa vị từ đầu tiên cũng thỏa vị từ thứ 2
- tương tự:
  - all(pig,big) (tương đương với $\forall$x pig(x) $\rightarrow$ big(x))
  - exists(pig,big) (tương đương với $\exists$x pig(x) AND big(x))

Cách biểu diễn vị từ

- Gilly swallowed a goldfish
  - swallowed(Gilly, goldfish)
- goldfish không phải là tên một đối tượng duy nhất như là Gilly

- Vấn đề

  Gilly swallowed a goldfish and Milly swallowed a goldfish

  Chuyển thành:

  swallowed(Gilly, goldfish) AND swallowed(Milly, goldfish)

  nhưng không phải cùng 1 goldfish …

Sử dụng lượng từ

- Gilly swallowed a goldfish
  - swallowed(Gilly, goldfish)
- Tốt hơn: ∃g goldfish(g) AND swallowed(Gilly, g)
- Hoặc sử dụng vị từ lượng từ:
  - exists(λg goldfish(g), λg swallowed(Gilly, g))
  - tương đương: exists(goldfish, swallowed(Gilly))
    - "trong tập cá có 1 con cá bị nuốt bởi Gilly"

Thời

- Gilly swallowed a goldfish
  - exists(goldfish, λg swallowed(Gilly, g))
- Cải tiến:
  - swallowed(Gilly, g) chuyển thành swallow(t, Gilly, g), t là thời gian
  - ∃t past(t) AND exists(λg goldfish(g), λg swallow(t, Gilly, g))
  - "Có lúc nào đó trong quá khứ, con cá là 1 trong các vật bị Gilly nuốt"

Các thuộc tính về sự kiện

- `Gilly swallowed a goldfish`
  - ∃t past(t) AND exists(goldfish, swallow(t, Gilly))
- Một phát biểu có thể có các thuộc tính khác nữa:
  - [Gilly] swallowed [a goldfish] [on a dare] [in a telephone booth] [with 30 other freshmen] [after many bottles of vodka had been consumed].
  - xác định ai? làm gì? tại sao? khi nào? (who what why when)
- Thay biến thời gian t với biến sự kiện e
  - ∃e past(e), act(e, swallowing), swallower(e, Gilly), exists(goldfish, swallowee(e)), exists(booth, location(e)), …
    - Có thể biểu diễn quá khứ bằng λe ∃t before(t,now), ended-at(e,t)

Trật tự của lượng từ

- Ví dụ
  - In this country a woman gives birth every 15 min. Our job is to find that woman and stop her.
  - ∃woman (∀15min gives-birth-during(woman, 15min))
  - ∀15min (∃woman gives-birth-during(15min, woman))

Các thuộc tính về ý chí

- Willy wants a unicorn
  - ∃e act(e,wanting), wanter(e,Willy), exists(unicorn, λu wantee(e,u))
    - "there is a unicorn u that Willy wants"
  - ∃e act(e,wanting), wanter(e,Willy), wantee(e, λu unicorn(u))
    - "Willy wants any entity u that satisfies the unicorn predicate"
- Willy wants Lilly to get married
  - ∃e present(e), act(e,wanting), wanter(e,Willy), wantee(e, λe' [act(e',marriage), marrier(e',Lilly)])
  - "Willy wants any event e' in which Lilly gets married"
  - Sentence doesn't claim that such an event exists
- Các động từ chỉ ý chí: hope, doubt, believe, …

Danh ngữ

- Expert λg expert(g)
- big fat expert λg big(g), fat(g), expert(g)
- Baltimore expert (white-collar expert, TV expert, …)
  - λg Related(Baltimore, g), expert(g) – expert from Baltimore

Câu

- Ý nghĩa của câu là gì?
  - Phụ thuộc vào dấu chấm câu.
  - Billy likes Lili. → assert(like(B,L))
  - Billy likes Lili? → ask(like(B,L))
    - Hoặc chính xác hơn, “Does Billy like Lili?”
  - Billy, like Lili! → command(like(B,L))

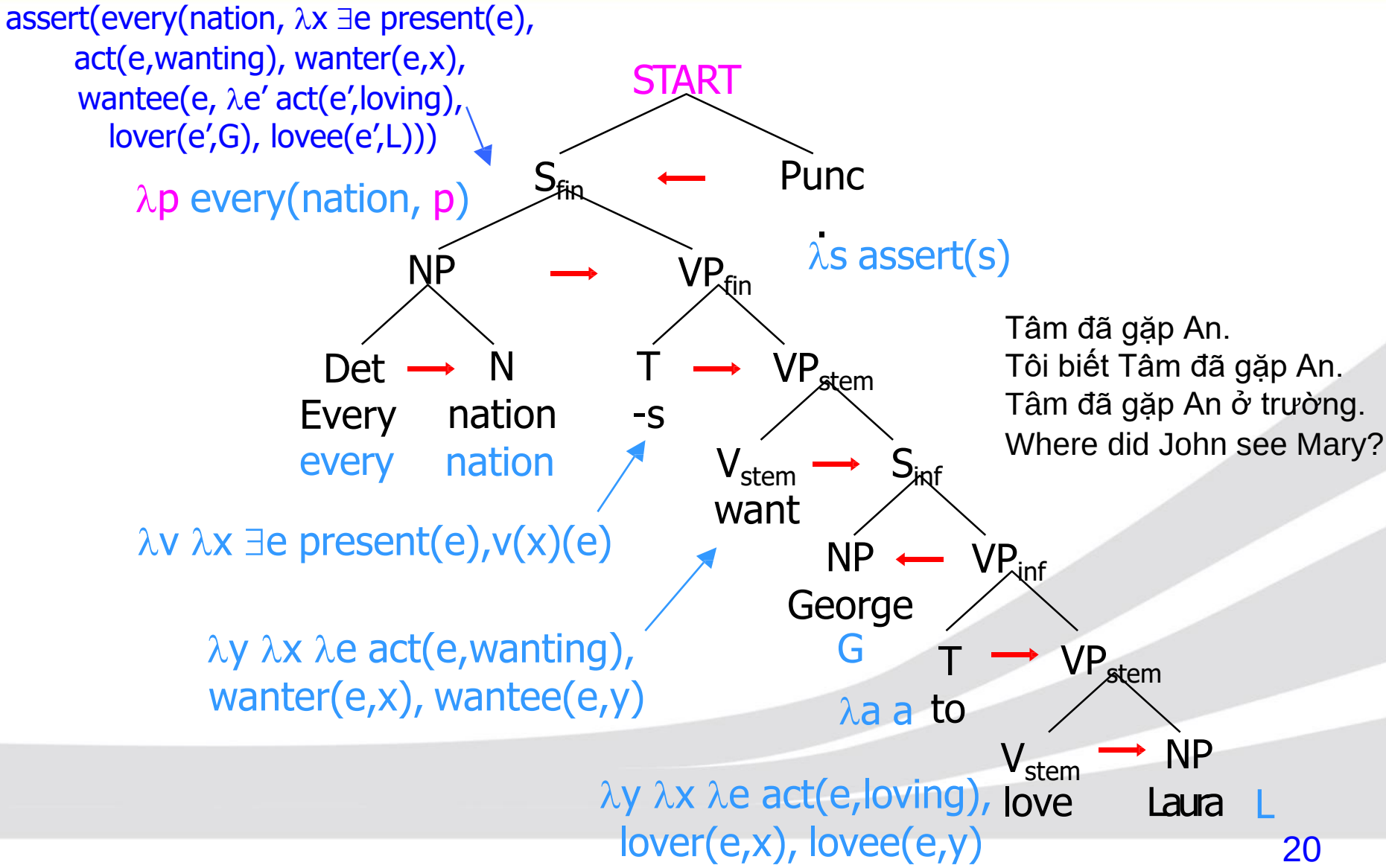
Câu

- What did Gilly swallow?
  - ask(λx ∃e past(e), act(e, swallowing),
    swallower(e, Gilly), swallowee(e,x))
- Eat your fish!Hearer
  - command(λf act(f, eating), eater(f, Hearer), eatee(f, λg fish(g)))
- I ate my fish.
  - assert(∃e past(e), act(e, eating), eater(e, Speaker), eatee(e, λg fish(g)))

Phân tích ngữ nghĩa câu

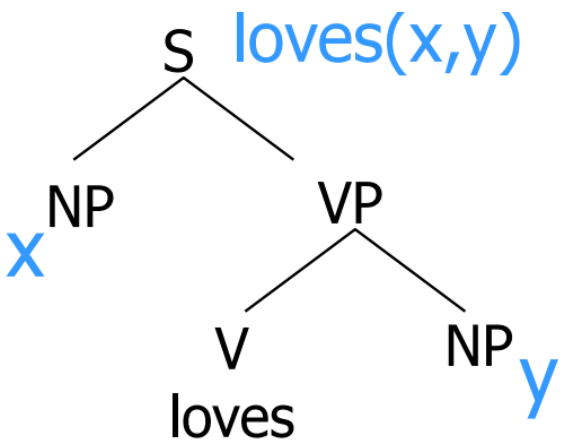
1. Phân tích cú pháp
2. Tìm ngữ nghĩa của từng từ
3. Xác định ngữ nghĩa cho mỗi thành phần ngữ pháp, thực hiện từ dưới lên

Ngữ nghĩa thành phần



Ngữ nghĩa thành phần

- Thêm thuộc tính "sem" cho mỗi luật phi ngữ cảnh
  - S → NP loves NP
  - S[sem=loves(x,y)] → NP[sem=x] loves NP[sem=y]
  - Nghĩa của S phụ thuộc vào nghĩa của NP
- TAG version:



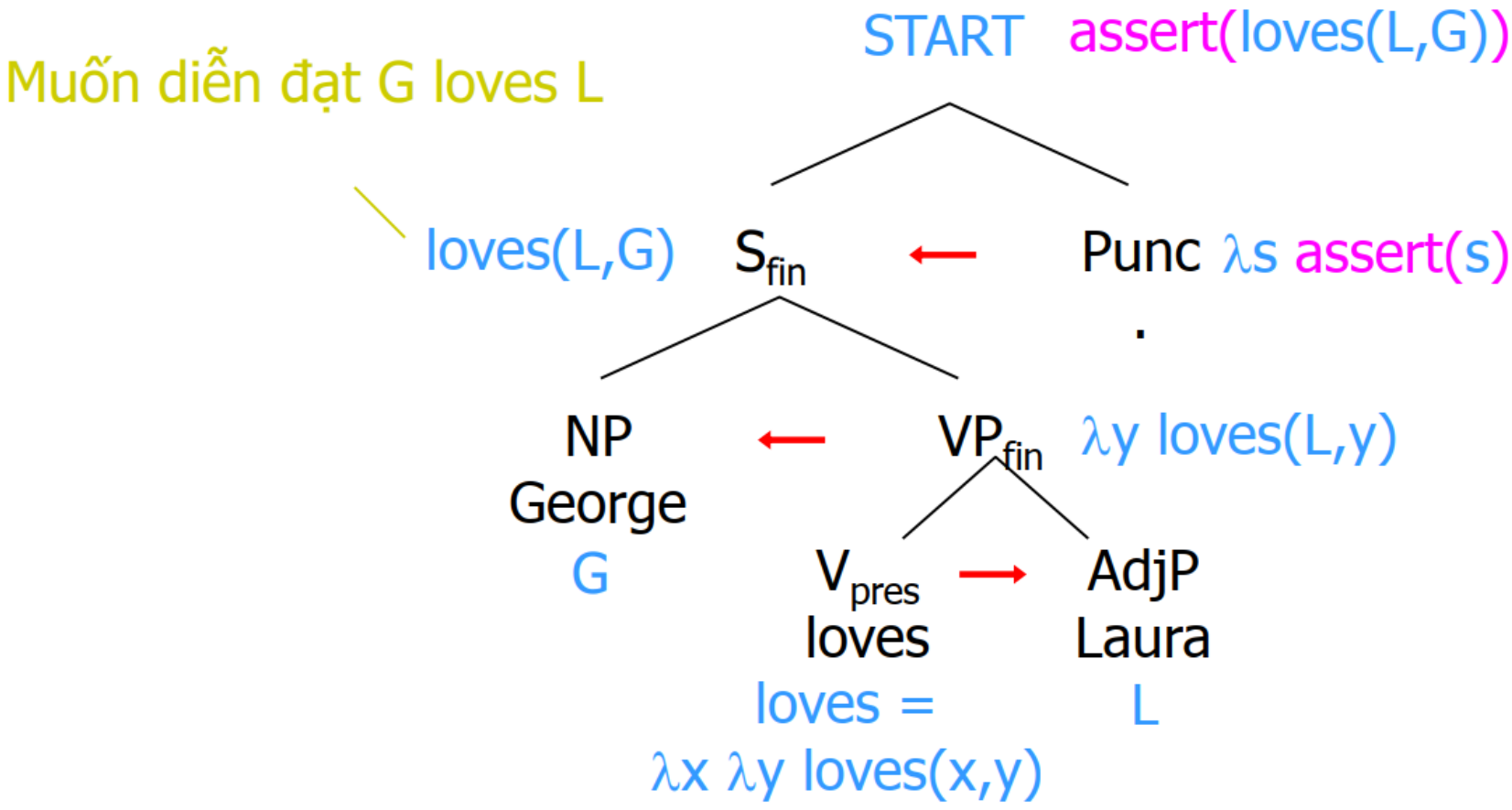
- Điền mẫu: S[sem=showflights(x,y)] →

  I want a flight from NP[sem=x] to NP[sem=y]

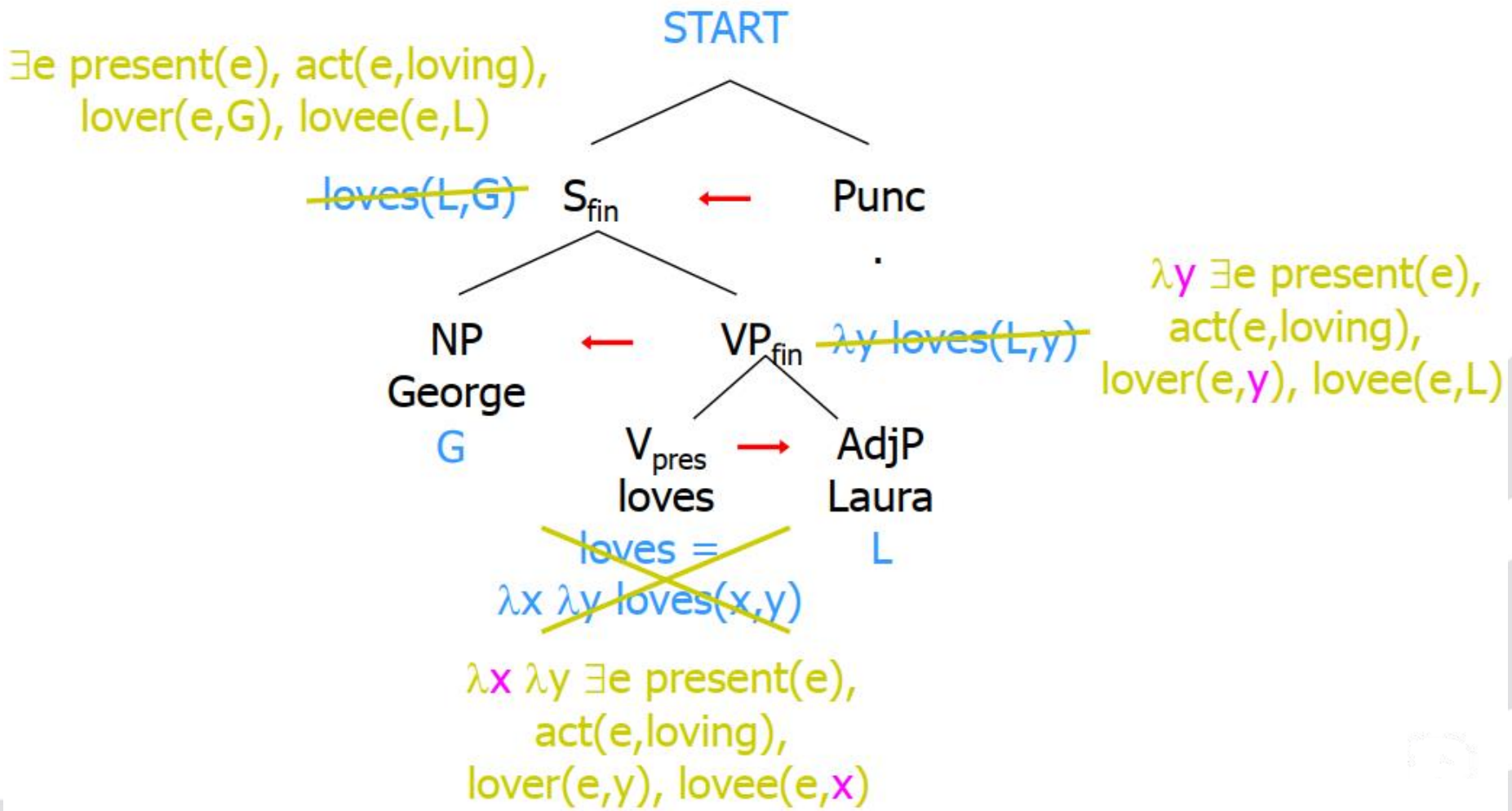
Ngữ nghĩa thành phần

- Thay S → NP loves NP
  - S[sem=loves(x,y)] → NP[sem=x] loves NP[sem=y]
- Luật tổng quát S → NP VP:
  - V[sem=loves] → loves
  - VP[sem=v(obj)] → V[sem=v] NP[sem=obj]
  - S[sem=vp(subj)] → NP[sem=subj] VP[sem=vp]
- `George loves Laura` **có** sem=loves(Laura)(George)
- Trong phần này, ta:
  - tính ngữ nghĩa từ dưới lên
  - Ngữ pháp ở dạng chuẩn Chomsky
  - Mỗi nút có 2 con: 1 hàm và 1 tham số
  - Để lấy ngữ nghĩa của nút, áp dụng hàm vào tham số

Ngữ nghĩa thành phần



Ngữ nghĩa thành phần



Cách biểu diễn ngữ nghĩa cơ bản

- Sử dụng “Event”
  - (EVENT :condition1 val1 :condition2 val2 …
    :condn valn)
- Ví dụ:
  - (see :agent John :patient Mary :tense past)

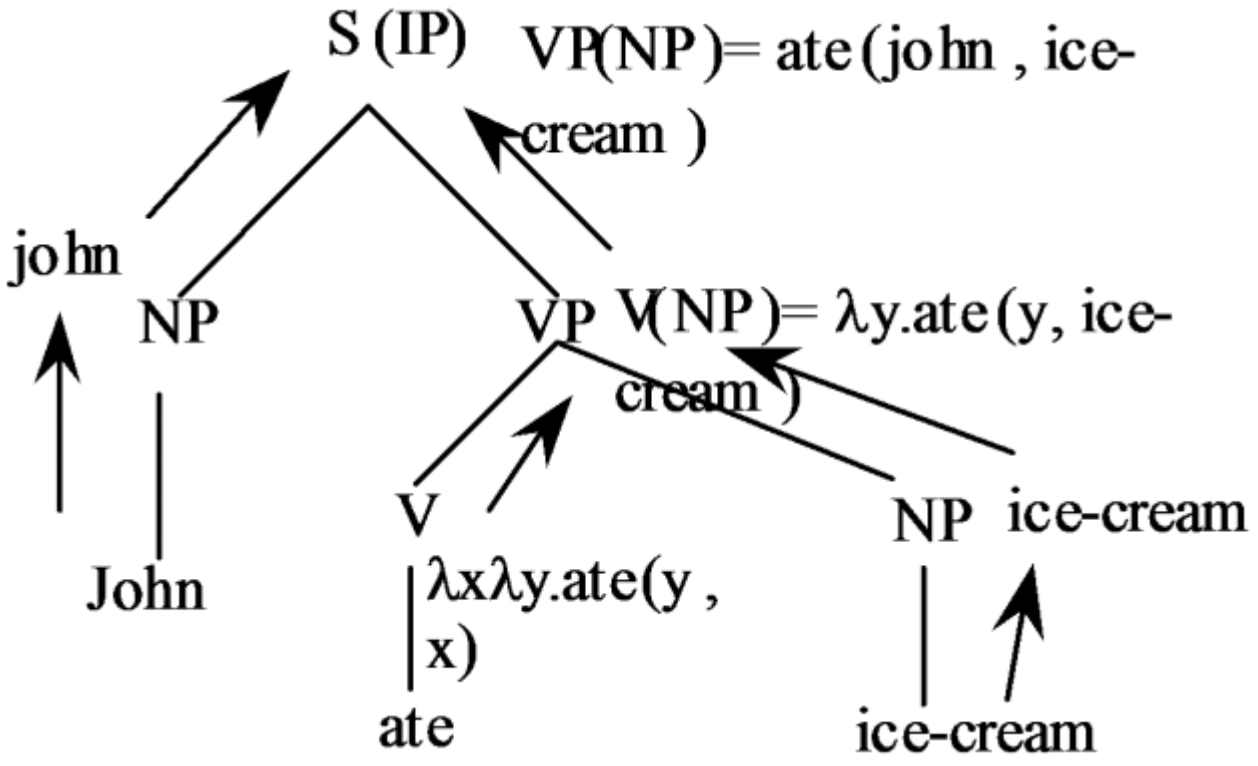
Luật cú pháp/ngữ nghĩa

Thành phần/luật	Dịch nghĩa
Verb *ate*	$\lambda x \lambda y.ate(y, x)$
N	N
V	V
S	S*= VP*(NP*)
NP	N*
VP	V*(NP*)

Ý nghĩa của câu

- λ form ứng với VP gắn với λ form đi với NP
- Từ là các giá trị
- Cho cây cú pháp, phân tích từ dưới lên để có ngữ nghĩa của câu *ate(John, ice-cream)*
- Vị từ này có thể được đánh giá dựa trên CSDL để trả về 1 giá trị hoặc T/F.

Dịch ngữ nghĩa



Ví dụ

- Phân tích ngữ nghĩa của các câu sau. Chỉ rõ tập luật ngữ nghĩa đã áp dụng (dạng VP[sem=v(obj)] → V[sem=v] NP[sem=obj]).
  - Tâm đã gặp An.
  - Tôi biết Tâm đã gặp An.
  - Tâm đã gặp An ở trường.

Cách thực hiện

- Ở mức cao nhất gọi đến VP. Giá trị VP này được xác định ở mức lá bằng cách sử dụng tham số NP
- Nói cách khác, để tìm ý nghĩa của câu, ta gọi VP sử dụng tham số là NP
- Tại nút lá, mỗi từ cũng đi kèm thêm một số thông tin ngữ nghĩa

Ứng dụng của phân tích ngữ nghĩa

```
(top-level)
Shall I clear the database? (y or n) y
>John saw Mary in the park.
OK.
>Where did John see Mary?
IN THE PARK.
>John gave Fido to Mary.
OK.
>Who gave John Fido?
I DON'T KNOW
>Who gave Mary Fido?
JOHN
>John saw Fido.
OK.
>Who did John see?
FIDO AND MARY
```